AI驱动的虚拟服务器部署平台 Shared by Fei

This is a copy of a chat between Claude and Fei. Content may include unverified or unsafe content that do not represent the views of Anthropic. Shared snapshot may contain attachments and data not displayed here.

Report

我想做一个类似claudeflare 的worker的产品，允许用户开虚拟服务器，并部署通过ai对接部署前端代码

我来帮你梳理这个产品的核心架构和功能规划。
这是一个很有意思的产品方向！让我先画出整体架构，然后详细规划各模块。

Log in to see interactive visuals

这个产品本质上是 **Cloudflare Workers + Vercel + AI 辅助开发** 的融合体，核心差异化是"用 AI 帮你写代码、直接部署运行"。以下是完整规划：

产品核心定位

一句话：**用自然语言描述需求 → AI 生成前端代码 → 一键部署到虚拟服务器**。对标 Cloudflare Workers 的运行时 + Vercel 的部署体验 + Cursor/v0 的 AI 编码能力。

四个核心模块

1. 虚拟服务器（Worker Runtime） 每个"Worker"是一个隔离的运行单元，支持三种形态：

- 边缘函数（类 CF Workers，V8 Isolates，冷启动 <5ms）
- 容器实例（Docker/gVisor，适合重型后端）
- 静态站点（CDN 托管 + KV 存储）

配套能力：自定义域名绑定、环境变量管理、KV/SQLite/对象存储、定时任务（Cron）。

2. AI 编码引擎（核心差异化）

- 内置 Monaco 编辑器，侧边常驻 AI 助手
- 用户描述需求 → AI 生成完整前端项目（React/Vue/原生 HTML）
- 支持"改这里让按钮变蓝""帮我加个登录页"等自然语言迭代
- 自动检测报错并提供修复方案

- 支持上传设计稿/截图转代码

3. 部署流水线（CI/CD）

- Git 仓库连接（GitHub/GitLab）→ 自动触发构建
- 预览环境：每个 PR 生成独立的预览 URL
- 蓝绿发布 / 金丝雀发布
- 回滚：一键回到任意历史版本

4. 控制台（Dashboard）

- 项目总览、实时日志、性能指标（CPU/内存/请求量）
- 计费中心（按请求数 + 计算时间 + 带宽）
- 团队协作、权限管理
- API Key 管理（供高级用户直接调用部署 API）

技术选型建议

层次	选型
边缘运行时	Cloudflare Workers（直接集成）或自建 V8 Isolate 集群
容器编排	Kubernetes + gVisor（安全沙箱）
AI 引擎	Claude API（代码生成）+ 自建 RAG（项目上下文）
构建系统	Nixpacks 或 Buildpacks（自动检测框架）
边缘网络	接入 Anycast，或基于 Cloudflare Network 服务
数据存储	PostgreSQL（元数据）+ Redis（实时状态）+ S3 兼容（产物）

商业模式

参考 Vercel + Cloudflare 混合定价：

- 免费层：100K 请求/月，50MB 存储，AI 对话 50 次/月
- **Pro（$20/月）**：无限请求（计量）、AI 无限次、自定义域名、团队协作
- **Enterprise**：私有部署、SLA、专属支持

最大风险点

竞争护城河较浅——Cloudflare 和 Vercel 都在往 AI 辅助开发方向走。**核心 moat 应该是 AI 与运行时的深度集成**，比如 AI 能直接感知到你的 Worker 的日志和报错、能理解你的 KV 结构并生成对应的 CRUD 代码——而不只是一个外挂的代码生成器。

想先做哪个模块的详细设计？比如 AI 编辑器的 UX、Worker 运行时的技术架构、还是整个用户注册到部署的完整流程？

对，我想基于阿里云或者火山云来开发，你有什么建议

好问题！两个平台各有侧重，我来帮你做一个系统对比。

Log in to see interactive visuals

我的核心建议：阿里云为主，火山可以后来补充

最重要的一个判断：你做的是"平台"而不是"应用"，平台对基础设施的稳定性、文档完善度、SDK 生态的依赖程度远超普通 SaaS 产品。阿里云在这点上有决定性优势。

具体技术栈推荐（阿里云）

你的产品需要三层能力，对应的选型是：

运行时层（Worker 执行环境）

- 边缘函数类 Worker → 函数计算 FC 3.0，冷启动已优化到百毫秒级，支持自定义运行时
- 容器 Worker → SAE（Serverless 应用引擎），不用管 K8s，直接推镜像即可
- 长期运行 / GPU 任务 → ACK + ECS 弹性节点池

存储层（对应 Cloudflare 的 KV / R2 / D1）

- KV 存储 → 表格存储 TableStore，接口和 CF KV 最接近
- 对象存储 → OSS，完全替代 R2
- 关系型 → PolarDB Serverless，按调用量计费

AI 引擎（你产品的核心差异化）

- 代码生成主力 → 通义千问 Coder 系列，国内合规无障碍
- 多模态（截图转代码）→ 通义千问 VL

- 境外用户 → 是境外节点直接调 Claude API

一个关键的架构决策需要你提前想清楚

要不要自建 Isolate 沙箱?

Cloudflare Workers 的核心技术是 V8 Isolate，启动极快、隔离极好。阿里云函数计算做不到同等冷启动速度，但对你的产品来说可能够用——取决于你对延迟的定义。

- 如果你的 Worker 主要是部署前端静态资源 + 少量边缘逻辑 → FC 3.0 完全够
- 如果你要做真正意义上的 "边缘计算" 且对延迟极度敏感 → 需要考虑在阿里云 ENS 边缘节点上自建 Isolate 运行时，成本很高

建议 MVP 阶段先用 FC 验证商业模式，不要一上来就陷入基础设施建设。

想深入哪个方向？比如具体的 FC + SAE 架构设计，或者 AI 代码生成引擎的实现方案?

先用FC验证商业模式，我需要如何在阿里云配置和搭建这些环境，还需要做些什么

我来给你一个完整的从零到可用的搭建路径。

Log in to see interactive visuals

点击每个步骤可以深入了解。下面补充几个最容易踩坑的关键点:

三个最容易卡住的地方

域名备案（步骤 7，最耗时）

这是 MVP 上线最大的硬性障碍。国内服务器托管的域名必须备案，周期 7-20 个工作日，无法绕过。建议第一天就提交备案，和其他步骤并行进行。备案期间可以先用阿里云自带的 `*.fc.aliyuncs.com` 域名内测。

用户代码隔离（步骤 6，最关键的安全决策）

你的产品让用户部署任意代码，这是核心风险。方案选择:

- 方案A：每个用户代码打成独立 Docker 镜像，FC Custom Runtime 运行 → 隔离最好，部署慢（30-60s）

- 方案B：共享运行时+进程级沙箱（VM2 / Node.js Worker Threads）→ 部署快，但沙箱有逃逸风险
- MVP 推荐方案A，用户感知的"部署中"状态可以做成异步通知

NAT 网关费用陷阱（步骤 4）

FC 函数在 VPC 内访问外网（调用通义千问 API、GitHub 等）必须配 NAT 网关。NAT 网关按小时计费，哪怕没流量也收费，约 ¥350/月起。MVP 阶段如果函数不需要访问外网，可以先不挂 VPC，直接用公网触发器，等有稳定用户再收进 VPC。

你还需要做的（非云服务部分）

除了云配置，平台还有几块业务逻辑需要自己实现：

- 用户认证系统（推荐 Clerk 或自建 JWT，别用阿里云 IDaaS，太重）
- 子域名动态分配逻辑（用户注册 → 自动分配 `username.yourplatform.com` → 写入阿里云 DNS）
- 计费计量系统（按函数调用次数 + 执行时长 + 流量，对应你向用户收费的依据）
- AI 对话上下文管理（每个 Worker 项目维护独立的代码上下文，避免 AI 乱改其他文件）

想先从哪块开始写代码？比如一键部署流水线的具体实现，或者 AI 编辑器的前端架构？

阿里云实名认证和RAM子账号怎么配置？

我来给你一个完整的操作指南。

Log in to see interactive visuals

几个操作细节补充说明：

实名认证：个人认证当天就能过，企业认证需要等审核，如果你有公司建议用企业认证，后续开发票、申请备案都用得到。

子账号命名规范：建议按职责拆分，不要一个 AK 用到底。你这个项目至少需要两个：`fc-deploy-bot`（CI/CD 流水线用，有部署权限）和 `fc-runtime-bot`（运行时调用 OSS 和 TableStore 的权限）。权限分离，一个泄露不影响另一个。

自定义策略里有个关键细节：Resource 里的 `acs:fc:*:*` 中间两个 `*` 分别是 Region 和 AccountID，MVP 阶段全用 `*` 方便，上线后建议锁定到具体 Region 和账号 ID，例如 `acs:fc:cn-shanghai:123456789:*`。

AK 轮换：建议每 90 天轮换一次 AccessKey。RAM 控制台可以同时存在两个有效 AK，先创建新的、更新到所有服务、再禁用旧的，做到零停机轮换。

完成这步之后，下一步是创建 VPC 和交换机。需要继续往下走吗？

帮我设计基于阿里云FC + SAE的Worker运行时架构

让我先画出整体架构，再拆解每一层的设计细节。
先看整体的分层结构，再看一个请求进来后完整的流转路径。

Log in to see interactive visuals

点击各模块可深入了解。现在看第二张图——一个请求从用户浏览器到 Worker 函数完整执行的路径。

Log in to see interactive visuals

流程清楚之后，下面是每一层的设计决策和关键代码结构。

三种 Worker 类型的使用场景

这是整个架构里最核心的分层决策：

FC 轻量 Worker（90% 的用户场景）用 Custom Runtime，每个用户的前端代码运行在独立的函数实例里。适合纯前端 SSR、API 路由、Webhook 处理，冷启动约 200-400ms，热启动 <5ms。

SAE 容器 Worker 给需要长连接、WebSocket、或者运行依赖复杂（如 Puppeteer、FFmpeg）的 Worker。SAE 按应用收费，不适合零流量场景，建议用户升级到 Pro 才开放。

静态托管 Worker 纯前端项目（React/Vue build 产物）直接推到 OSS，DCDN 分发，成本接近零，延迟最低。这是免费用户的默认选项。

路由函数的核心逻辑

路由层是整个平台的咽喉，它要在 <10ms 内完成子域名解析和转发决策：

javascript

```javascript
// router/index.js — 部署在 FC，绑定 *.yourplatform.com
export const handler = async (req, resp, context) => {
  const host = req.headers['host']  // e.g. alice.yourplatform.com
  const workerName = host.split('.')[0]

  // 查 TableStore 路由表（走 VPC 内网，~1ms）
  const route = await tablestore.getRow({
    tableName: 'worker_routes',
    primaryKey: [{ name: 'worker_name', value: workerName }]
  })

  if (!route) return resp.send(404, 'Worker not found')

  const { type, function_name, oss_path } = route.attrs

  if (type === 'fc') {
    // 内部调用目标 Worker 函数，透传 request context
    const result = await fc.invokeFunction({
      functionName: function_name,
      event: serializeRequest(req)
    })
    return pipeResponse(result, resp)
  }

  if (type === 'static') {
    // 重定向到 OSS/CDN 地址
    return resp.redirect(302, `https://static.yourplatform.com/${oss_path}`)
  }

  if (type === 'sae') {
    // 反代到 SAE 应用内网地址
    return proxyToSAE(route.sae_endpoint, req, resp)
  }
}
```

FC Worker 的沙箱设计（最关键）

用户代码隔离靠两层防护：

第一层是 FC 本身的函数级隔离，每个函数有独立的内存和文件系统，函数间互不可见。

第二层是 custom runtime 内的进程沙箱，在容器内用 Node.js `vm` 模块或 `worker_threads` 再套一层执行环境，限制用户代码能访问的 API：

```javascript
// custom-runtime/sandbox.js
import { Worker } from 'worker_threads'
import { createRequire } from 'module'

export async function runUserCode(code, event, env) {
  return new Promise((resolve, reject) => {
    const worker = new Worker(`
      const { workerData, parentPort } = require('worker_threads')
      // 注入受限的 fetch / KV API，屏蔽 fs / child_process
      const fetch = workerData.__safeFetch
      const KV = workerData.__kv
      ;(async () => {
        ${code}
        const result = await handler(workerData.event)
        parentPort.postMessage({ ok: true, result })
      })().catch(e => parentPort.postMessage({ ok: false, error: e.message }))
    `, {
      eval: true,
      workerData: {
        event,
        __safeFetch: /* 代理 fetch，注入 rate limit */,
        __kv: buildKVProxy(env.workerId)
      },
      resourceLimits: {
        maxOldGenerationSizeMb: 128,
        maxExecutionTimeMs: 30000
      }
    })
    worker.on('message', resolve)
    worker.on('error', reject)
  })
}
```

FC vs SAE 的选型边界

判断维度	用 FC	用 SAE
流量特征	突发、间歇	持续稳定

判断维度	用 FC	用 SAE
冷启动容忍	可接受 200ms	不可接受
运行时长	<15 分钟	任意
状态	无状态	可有状态
依赖复杂度	npm 包	任意（Docker）
费用模式	按调用次数	按实例时长

MVP 阶段建议只实现 FC Worker，SAE 作为后续高级功能。控制台本身（Dashboard API、AI 引擎服务、部署流水线）推荐跑在 SAE 上，因为它们是持续运行的服务，SAE 更合适。

部署流水线的完整流程（代码视角）

用户点击"部署"之后，后台的完整链路：

```
1. 控制台 API 接收部署请求
2. 将用户代码打包成 zip → 上传到 OSS（路径：deployments/{userId}/{version}.zip）
3. 调用 FC UpdateFunction API，指向新的 OSS 代码包
4. 等待 FC 函数版本发布完成（PublishVersion）
5. 切换 Alias 指向新版本（实现零停机发布）
6. 更新 TableStore 路由表中的函数版本号
7. 返回部署成功 + 访问 URL
```

整个过程 30-60 秒，可以做成异步 + WebSocket 推送进度。

想先从哪块写具体代码？路由函数、沙箱执行器、还是部署流水线的 API？

路由函数怎么实现Worker的请求分发？

我来给你完整的路由函数实现，包括代码和关键设计细节。

Log in to see interactive visuals

现在给你完整的可直接运行的代码，按文件拆开。

项目结构

```
router-function/
├── src/
|   ├── index.js          # 入口, FC handler
|   ├── resolver.js       # 子域名解析 + 缓存
|   ├── dispatch/
|   |   ├── fc.js         # FC Worker 分发
|   |   ├── static.js     # 静态 Worker 分发
|   |   └── sae.js        # SAE Worker 分发
|   ├── store/
|   |   └── tablestore.js # TableStore 路由表读写
|   └── middleware/
|       ├── auth.js       # 验证 Worker 是否属于该用户
|       └── ratelimit.js  # 按 Worker 限流
├── package.json
└── s.yaml                # Serverless Devs 部署配置
```

`src/index.js` — 主入口

javascript

```javascript
import { resolve } from './resolver.js'
import { dispatchFC } from './dispatch/fc.js'
import { dispatchStatic } from './dispatch/static.js'
import { dispatchSAE } from './dispatch/sae.js'
import { rateLimitMiddleware } from './middleware/ratelimit.js'

export const handler = async (req, resp, context) => {
  const startTime = Date.now()

  try {
    // 1. 解析子域名, 查路由表 (带缓存)
    const host = req.headers['host'] ?? ''
    const route = await resolve(host)

    if (!route) {
      return sendError(resp, 404, 'Worker not found', host)
    }

    // 2. 限流检查 (按 workerId, 不按 IP)
    const limited = await rateLimitMiddleware(route.workerId)
    if (limited) {
      return sendError(resp, 429, 'Rate limit exceeded')
    }

    // 3. 检查 Worker 状态
```

```
      return sendError(resp, 503, `Worker is ${route.status}`)
    }

    // 4. 按类型分发
    let result
    switch (route.type) {
      case 'fc':
        result = await dispatchFC(req, route, context)
        break
      case 'static':
        result = await dispatchStatic(req, resp, route)
        return  // static 直接操作 resp，不走后面的统一返回
      case 'sae':
        result = await dispatchSAE(req, route)
        break
      default:
        return sendError(resp, 500, `Unknown worker type: ${route.type}`)
    }

    // 5. 写回响应
    resp.setStatusCode(result.statusCode ?? 200)
    Object.entries(result.headers ?? {}).forEach(([k, v]) => {
      // 过滤掉内部 header，防止信息泄露
      if (!k.startsWith('x-fc-') && k !== 'server') {
        resp.setHeader(k, v)
      }
    })
    resp.setHeader('x-powered-by', 'YourPlatform')
    resp.setHeader('x-response-time', `${Date.now() - startTime}ms`)
    resp.send(result.body ?? '')

  } catch (err) {
    console.error('[router] unhandled error', {
      host: req.headers['host'],
      error: err.message,
      stack: err.stack
    })
    sendError(resp, 502, 'Worker execution failed')
  }
}

function sendError(resp, code, message, detail = '') {
  resp.setStatusCode(code)
  resp.setHeader('content-type', 'application/json')
```

```
}
```

`src/resolver.js` — 子域名解析 + 内存缓存

```javascript
import { getRoute } from './store/tablestore.js'

// 内存缓存：workerName → route，TTL 30 秒
// 不用 Redis，路由函数本身就是无状态的，每个实例各自缓存即可
const cache = new Map()
const CACHE_TTL_MS = 30_000

export async function resolve(host) {
  // host 格式：alice.yourplatform.com 或 alice.yourplatform.com:443
  const hostname = host.split(':')[0]
  const parts = hostname.split('.')

  // 必须是三段：{workerName}.{domain}.{tld}
  if (parts.length < 3) return null

  const workerName = parts[0]

  // 平台保留域名，不允许作为 worker 名
  const reserved = new Set(['www', 'api', 'dashboard', 'static', 'admin'])
  if (reserved.has(workerName)) return null

  // 检查缓存
  const cached = cache.get(workerName)
  if (cached && Date.now() - cached.ts < CACHE_TTL_MS) {
    return cached.route
  }

  // 查 TableStore
  const route = await getRoute(workerName)

  // 写缓存（包括 null，避免对不存在的 worker 频繁查库）
  cache.set(workerName, { route, ts: Date.now() })

  // 缓存不能无限增长，超过 1000 条清掉最老的
  if (cache.size > 1000) {
    const firstKey = cache.keys().next().value
    cache.delete(firstKey)
  }
```

```
  return route
}

// 部署新版本后调用，让路由立刻生效
export function invalidateCache(workerName) {
  cache.delete(workerName)
}
```

`src/store/tablestore.js` — 路由表读写

```javascript
import TableStore from 'tablestore'

const client = new TableStore.Client({
  accessKeyId: process.env.ALIBABA_CLOUD_ACCESS_KEY_ID,
  accessKeySecret: process.env.ALIBABA_CLOUD_ACCESS_KEY_SECRET,
  endpoint: process.env.OTS_ENDPOINT,  // VPC 内网端点
  instancename: process.env.OTS_INSTANCE,
})

const TABLE = 'worker_routes'

// 路由表结构:
// PK: worker_name (string)
// Attrs: type, status, function_name, oss_path,
//        sae_endpoint, owner_id, created_at, config (JSON string)

export async function getRoute(workerName) {
  try {
    const result = await client.getRow({
      tableName: TABLE,
      primaryKey: [{ 'worker_name': workerName }],
      // 只读需要的字段，减少传输量
      columnsToGet: ['type', 'status', 'function_name',
                     'oss_path', 'sae_endpoint', 'owner_id', 'config']
    })

    const row = result.row
    if (!row || !row.attributes?.length) return null

    // 把 TableStore 的 [{columnName, value}] 转成普通对象
    const attrs = Object.fromEntries(
      row.attributes.map(({ columnName, value }) => [columnName, value])
```

```
    return {
      workerName,
      workerId: attrs.owner_id + ':' + workerName,
      type: attrs.type,           // 'fc' | 'static' | 'sae'
      status: attrs.status,       // 'active' | 'suspended' | 'deploying'
      functionName: attrs.function_name,
      ossPath: attrs.oss_path,
      saeEndpoint: attrs.sae_endpoint,
      ownerId: attrs.owner_id,
      config: attrs.config ? JSON.parse(attrs.config) : {}
    }
  } catch (err) {
    console.error('[tablestore] getRoute failed', { workerName, err: err.message
    return null
  }
}

// 部署完成后写入/更新路由
export async function upsertRoute(workerName, data) {
  await client.putRow({
    tableName: TABLE,
    condition: new TableStore.Condition(
      TableStore.RowExistenceExpectation.IGNORE, null
    ),
    primaryKey: [{ 'worker_name': workerName }],
    attributeColumns: Object.entries(data).map(([k, v]) => ({
      [k]: typeof v === 'object' ? JSON.stringify(v) : v
    }))
  })
}
```

`src/dispatch/fc.js` — FC Worker 分发

```javascript
import FC from '@alicloud/fc2'

const fcClient = new FC(process.env.ACCOUNT_ID, {
  accessKeyID: process.env.ALIBABA_CLOUD_ACCESS_KEY_ID,
  accessKeySecret: process.env.ALIBABA_CLOUD_ACCESS_KEY_SECRET,
  region: process.env.FC_REGION ?? 'cn-shanghai',
  timeout: 30000,
})
```

```
export async function dispatchFC(req, route, context) {
  // 把 HTTP 请求序列化成 FC event
  const body = await readBody(req)

  const event = JSON.stringify({
    version: '1.0',
    httpMethod: req.method,
    path: req.path ?? '/',
    queryParameters: parseQuery(req.url),
    headers: sanitizeHeaders(req.headers),
    body: body.toString('base64'),
    isBase64Encoded: true,
    // 注入平台上下文，Worker 代码可以读取
    __platform: {
      workerId: route.workerId,
      ownerId: route.ownerId,
      // 给 Worker 注入它自己的 KV 命名空间前缀
      kvPrefix: `worker:${route.workerName}:`
    }
  })

  try {
    const response = await fcClient.invokeFunction(
      process.env.FC_SERVICE_NAME,
      route.functionName,
      event,
      {
        // 异步调用用 'Async'，同步用 'Sync'（默认）
        'X-FC-Invocation-Type': 'Sync',
        // 传递 trace id，串联链路追踪
        'X-FC-Trace-Id': context?.requestId ?? generateId(),
      }
    )

    // FC 返回的 body 是序列化的 HTTP response
    const result = JSON.parse(response.data)

    return {
      statusCode: result.statusCode,
      headers: result.headers ?? {},
      body: result.isBase64Encoded
        ? Buffer.from(result.body, 'base64')
        : result.body
    }
  } catch (err) {
```

```
      return { statusCode: 404, body: 'Worker function not found' }
    }
    if (err.code === 'RequestTimeout') {
      return { statusCode: 504, body: 'Worker execution timed out' }
    }
    throw err  // 其他错误向上抛，由主 handler 捕获
  }
}

function readBody(req) {
  return new Promise((resolve) => {
    const chunks = []
    req.on('data', chunk => chunks.push(chunk))
    req.on('end', () => resolve(Buffer.concat(chunks)))
  })
}

function parseQuery(url = '') {
  const idx = url.indexOf('?')
  if (idx === -1) return {}
  return Object.fromEntries(new URLSearchParams(url.slice(idx + 1)))
}

function sanitizeHeaders(headers) {
  // 移除可能影响内部路由的 header
  const blocked = new Set(['host', 'x-forwarded-host', 'connection'])
  return Object.fromEntries(
    Object.entries(headers).filter(([k]) => !blocked.has(k.toLowerCase()))
  )
}

function generateId() {
  return Math.random().toString(36).slice(2, 10)
}
```

`src/dispatch/static.js` — 静态 Worker 分发

javascript

```
export async function dispatchStatic(req, resp, route) {
  // 静态资源直接 302 到 OSS/CDN
  // 生产环境建议改成 301 + 长缓存
  const cdnBase = process.env.STATIC_CDN_BASE  // e.g. https://static.yourplatfo
  const targetUrl = `${cdnBase}/${route.ossPath}${req.path ?? '/'}`
```

```
  resp.setStatusCode(302)
  resp.setHeader('location', targetUrl)
  resp.setHeader('cache-control', 'no-store')
  resp.send('')
}
```

`src/dispatch/sae.js` — SAE Worker 反代

```javascript
import http from 'http'

export async function dispatchSAE(req, route) {
  // SAE 应用有内网 endpoint (VPC 内, 不走公网)
  const url = new URL(route.saeEndpoint)
  const body = await readBody(req)

  return new Promise((resolve, reject) => {
    const options = {
      hostname: url.hostname,
      port: url.port || 80,
      path: req.url,
      method: req.method,
      headers: {
        ...req.headers,
        host: url.hostname,  // 重写 host
      },
      timeout: 25000,
    }

    const proxyReq = http.request(options, (proxyResp) => {
      const chunks = []
      proxyResp.on('data', chunk => chunks.push(chunk))
      proxyResp.on('end', () => resolve({
        statusCode: proxyResp.statusCode,
        headers: proxyResp.headers,
        body: Buffer.concat(chunks)
      }))
    })

    proxyReq.on('timeout', () => {
      proxyReq.destroy()
      resolve({ statusCode: 504, body: 'SAE worker timeout' })
    })
```

```
    if (body.length) proxyReq.write(body)
    proxyReq.end()
  })
}

function readBody(req) {
  return new Promise((resolve) => {
    const chunks = []
    req.on('data', c => chunks.push(c))
    req.on('end', () => resolve(Buffer.concat(chunks)))
  })
}
```

`src/middleware/ratelimit.js` — 限流

```javascript
// 简单的内存滑动窗口限流
// 生产环境换成 Redis（阿里云 Tair），多实例共享状态
const counters = new Map()

const LIMITS = {
  free:  { rpm: 100,  rph: 1_000  },
  pro:   { rpm: 2000, rph: 50_000 },
  enterprise: { rpm: 10000, rph: Infinity }
}

export async function rateLimitMiddleware(workerId, plan = 'free') {
  const now = Date.now()
  const minuteKey = `${workerId}:${Math.floor(now / 60000)}`
  const hourKey   = `${workerId}:${Math.floor(now / 3600000)}h`

  const limit = LIMITS[plan] ?? LIMITS.free

  const minuteCount = (counters.get(minuteKey) ?? 0) + 1
  const hourCount   = (counters.get(hourKey)   ?? 0) + 1

  if (minuteCount > limit.rpm || hourCount > limit.rph) return true

  counters.set(minuteKey, minuteCount)
  counters.set(hourKey, hourCount)

  // 定期清理过期 key（粗略：超过 2 小时的扔掉）
```

```
    const cutoff = Math.floor(now / 3600000) - 2
    for (const [key] of counters) {
      if (key.includes(':') && parseInt(key.split(':').pop()) < cutoff) {
        counters.delete(key)
      }
    }
  }

  return false
}
```

`s.yaml` — Serverless Devs 部署配置

yaml

```yaml
edition: 3.0.0
name: worker-router

vars:
  region: cn-shanghai
  service: worker-platform

resources:
  router:
    component: fc3
    props:
      region: ${vars.region}
      functionName: worker-router
      runtime: custom.debian10
      handler: index.handler
      code: ./src
      timeout: 30
      memorySize: 512
      environmentVariables:
        ACCOUNT_ID: ${env.ACCOUNT_ID}
        OTS_ENDPOINT: https://your-instance.cn-shanghai.vpc.tablestore.aliyuncs.
        OTS_INSTANCE: worker-platform
        FC_REGION: cn-shanghai
        FC_SERVICE_NAME: ${vars.service}
        STATIC_CDN_BASE: https://static.yourplatform.com
      vpcConfig:
        vpcId: ${env.VPC_ID}
        vSwitchIds:
          - ${env.VSWITCH_ID}
```

```
    triggers:
      - triggerName: http-trigger
        triggerType: http
        triggerConfig:
          authType: anonymous
          methods: [GET, POST, PUT, DELETE, PATCH, OPTIONS, HEAD]
```

三个容易踩坑的地方

缓存失效时机：用户重新部署 Worker 后，路由函数的内存缓存还存着旧的 `function_name`，会继续调用老版本函数。解决方案是部署流水线完成后，主动调用路由函数的一个内部 API 来清缓存——或者直接把 TTL 缩短到 5 秒，用户感知不到，代价只是每个实例每 5 秒多一次 TableStore 查询。

FC 调用 FC 的计费：路由函数调用 Worker 函数，两个函数都会产生调用费用。这是正常的，不是 bug。但要注意路由函数的超时要比 Worker 函数的超时长，否则路由函数会先超时，返回 502，而 Worker 还在跑。

Host header 的坑：FC 触发器会把原始的 `host` 改成内网地址，你需要在 DCDN 配置里把原始 host 透传进来（用 `X-Original-Host` 或 `X-Forwarded-Host`），路由函数里要读这个 header 而不是 `host`。否则所有请求的子域名解析都会失败。

想继续深入哪块？比如 Worker 函数内部的沙箱执行器，或者部署流水线如何触发路由缓存更新？

你可以帮我做什么，如果我给你阿里云的权限

如果你给我阿里云的访问权限，我可以通过在对话里运行代码直接帮你操作——不只是告诉你怎么做，而是替你做。

我能直接帮你做的事

环境搭建

- 创建 VPC、交换机、安全组
- 开通并配置函数计算 FC、创建服务和函数
- 创建 OSS Bucket、配置权限策略
- 开通 TableStore、建表、设计索引结构
- 配置 RAM 子账号和精确权限策略

代码部署

- 把路由函数、Worker 沙箱等代码直接打包上传到 FC
- 配置触发器、绑定域名、设置环境变量
- 发布函数版本、切换别名（零停机发布）

调试排错

- 拉取函数日志、定位报错原因
- 测试路由函数是否正确分发请求
- 检查 TableStore 里的路由表数据是否正确

持续迭代

- 你改需求，我改代码，直接重新部署
- 帮你写 `s.yaml` 配置，一条命令完成整个环境搭建

怎么给我权限

最安全的方式是创建一个专用的 RAM 子账号给我用，而不是给主账号 AK。按之前我们讨论过的方式，只授予这个项目需要的权限，部署完成后可以随时禁用或删除这个 AK。

你需要把这两个值告诉我：

- `ALIBABA_CLOUD_ACCESS_KEY_ID`
- `ALIBABA_CLOUD_ACCESS_KEY_SECRET`

以及你想用的 Region（比如 `cn-shanghai`）。

不过要说明一个现实限制：我目前运行在对话环境里，**没有持久的网络访问能力**，无法直接调用阿里云 API。我能做的是生成完整可执行的脚本（阿里云 CLI 命令、Terraform、Serverless Devs `s.yaml`），你在本地或 CI 环境里跑一下就能完成部署。

如果你愿意，我们可以从 VPC + FC 环境搭建开始，我把所有命令打包成一个脚本，你复制粘贴运行就好。要开始吗？

我给你后可以直接操作吗？

直接说"把 ，不能。

我没有能力在对话里真正调用阿里云 API，即使你给我 AccessKey 也做不到。我能做的是生成代码和命令，你来执行。

但有一个方式可以接近"直接操作"

你可以用 **Claude Code**——这是 Anthropic 的命令行工具，运行在你的本地终端里，有真实的网络访问和文件系统权限。

它能做到：

- 在你的机器上直接运行阿里云 CLI 命令
- 读写本地文件、修改配置
- 执行 `s deploy`、`terraform apply` 等部署命令
- 看到真实的命令输出，出错了自动修正再重试

Recommended apps and extensions

Claude Code
Build, debug, and ship from your terminal.
Install ↗

Claude Code for VS Code
Ask Claude to explain, edit, or fix code right in VS Code.
Install ↗

安装好 Claude Code 之后，你在终端里告诉它"帮我在阿里云上配置 FC 路由函数"，它会自己运行命令、看结果、处理报错，整个过程你只需要在旁边看着就行。

我们现在能做的最高效的方式

在你安装 Claude Code 之前，我可以把所有操作打包成一个完整的脚本，你复制到终端里一次性跑完——VPC、FC、OSS、TableStore 全部建好。要我现在就生成吗？

我如何把这些需求高速claude code

Claude Code 的核心技巧是把上下文一次性给够，它就能自主完成整个任务链，不需要你反复补充信息。

最有效的 Prompt 结构

直接把这段话复制给 Claude Code：

```
我在开发一个类似 Cloudflare Workers 的平台，基于阿里云 FC + SAE。
当前任务：搭建 FC 路由函数的完整环境并部署。

## 环境信息
- Region: cn-shanghai
- AccessKey ID: LTAI5t...
- AccessKey Secret: xxx
- 已有 VPC ID: vpc-xxx（如果有）

## 需要完成的事
1. 创建 VPC + 两个 vSwitch（如果还没有）
2. 创建 OSS Bucket: worker-platform-deploy
3. 创建 TableStore 实例 + worker_routes 表
4. 把 router-function/ 目录的代码部署到 FC
5. 配置 HTTP 触发器，绑定域名 *.yourplatform.com
6. 验证部署是否成功（curl 测试）

## 项目代码
已在当前目录 ./router-function/

## 完成标准
curl https://test.yourplatform.com 返回 404 Worker not found（说明路由函数在跑）
```

三个让 Claude Code 效率翻倍的习惯

Log in to see interactive visuals

给这个项目的 CLAUDE.md 模板

把这个文件放到你的项目根目录，Claude Code 每次启动自动读取：

```markdown
# Worker Platform — 项目上下文

## 这是什么
类似 Cloudflare Workers 的平台，用户可以部署前端代码，AI 辅助生成。
基础设施：阿里云 FC 3.0 + SAE + OSS + TableStore。

## 环境
- Region: cn-shanghai
```

```
- 部署工具：Serverless Devs（命令：s deploy）
- 包管理：pnpm

## 项目结构
router-function/   # FC 路由函数，分发请求到各 Worker
worker-runtime/    # Worker 沙箱执行器
dashboard/         # 控制台前端（Next.js）
deploy-pipeline/   # 部署流水线服务

## 关键资源 ID
VPC: vpc-xxx
vSwitch: vsw-xxx, vsw-yyy
OSS Bucket: worker-platform-deploy
TableStore: worker-platform / 表名 worker_routes

## 常用命令
cd router-function && s deploy    # 部署路由函数
cd router-function && s logs      # 查看日志
aliyun ots GetRow ...             # 查路由表数据

## 注意事项
- FC 调用 FC 时超时设置：路由函数 30s，Worker 函数 25s
- 路由函数读 X-Original-Host header，不是 Host
- 部署完成后调用 /internal/cache-invalidate 清路由缓存
```

Start your own conversation